TrackB 기술 문서

보험상품 데이터 수집 및 구조화 파이프라인 - Playwright, pdfplumber, LLM 파싱

목차

1. 파이프라인 개요 및 데이터 흐름
2. 프로젝트 파일 구조
3. Playwright 웹 자동화 및 스크래핑
4. pdfplumber PDF 텍스트 추출
5. Pydantic JSON 스키마 설계
6. LLM 기반 구조화 파싱 (OpenRouter)
7. JSON 후처리 및 검증
8. 실행 환경 설정 및 명령어

1 파이프라인 개요 및 데이터 흐름

1.1 전체 데이터 흐름

웹사이트 → Scraping → Raw Data → Parsing → Structured JSON

1.2 3단계 프로세스

단계	파일	입력	출력	핵심 도구
Step 1 메타데이터 수집	`kb_export_rows.py`	KB생명 웹페이지	CSV (상품 목록)	Playwright
Step 2 PDF 다운로드	`kb_download_one.py`	상품 목록 페이지	PDF 파일들	Playwright
Step 3 파싱 및 구조화	`parse_one_pdf.py`	PDF 파일	JSON (구조화 데이터)	pdfplumber, OpenAI, Pydantic

1.3 데이터 유형별 처리 분기

```
IF 데이터가 "동적 UI 기반 (입력 → 결과)"
    → Playwright 사용 (브라우저 자동화)

ELSE IF 데이터가 "정적 HTML"
    → requests + BeautifulSoup

ELSE IF 데이터가 "PDF 문서"
    → PDF 다운로드 → pdfplumber로 텍스트 추출 → LLM 파싱
```

> **왜 Playwright인가?**
> KB생명 공시실은 JavaScript로 동적 렌더링되는 SPA(Single Page Application)입니다. 단순 HTTP 요청(requests)으로는 데이터를 가져올 수 없고, 실제 브라우저처럼 JavaScript를 실행해야 합니다. Playwright는 Chromium 브라우저를 프로그래밍 방식으로 제어하여 이를 해결합니다.

2 프로젝트 파일 구조

```
TrackB/
│
├── .env # API 키 저장 (Git 제외)
├── .gitignore # Git 추적 제외 파일 목록
│
├── kb_export_rows.py # [Step 1] 상품 메타데이터 수집
├── kb_download_one.py # [Step 2] 최신 상품 PDF 다운로드
├── parse_one_pdf.py # [Step 3] PDF 파싱 + LLM 구조화
│
├── kb_rows_metadata.csv # Step 1 결과: 상품 목록 CSV
│
├── downloads/ # 다운로드된 PDF (전체 버전)
├── downloads_latest/ # 최신 상품 PDF만
│   └── *.pdf
│
└── outputs/ # Step 3 결과물
    ├── extracted_text_preview.txt # PDF에서 추출한 원본 텍스트
    ├── raw_llm_output.txt # LLM 원본 응답 (디버깅용)
    └── parsed_insurance.json # 최종 구조화 JSON
```

2.1 핵심 파일 역할

1. `kb_export_rows.py` **Playwright**
   - KB생명 상품목록 페이지 크롤링
   - 상품명, 판매기간, 다운로드 버튼 수 등 메타데이터 추출
   - 결과를 CSV로 저장
2. `kb_download_one.py` **Playwright**
   - 현재 판매 중인 상품(종료일 없음) 필터링
   - 상품설명서/약관 PDF 자동 다운로드
   - 파일명에 상품명, 판매기간 포함
3. `parse_one_pdf.py` **pdfplumber** **OpenAI** **Pydantic**
   - PDF에서 텍스트 추출 (페이지별 마킹)
   - LLM(GPT-4o-mini)에게 구조화 요청
   - Pydantic으로 스키마 검증
   - 최종 JSON 저장

3 Playwright 웹 자동화 및 스크래핑

3.1 Playwright란?

Microsoft가 개발한 브라우저 자동화 도구로, Chromium/Firefox/WebKit을 프로그래밍 방식으로 제어합니다.

장점

- JavaScript 렌더링 완벽 지원
- 파일 다운로드 자동 처리
- 동적 UI 상호작용 가능
- 헤드리스/유헤드 모드 선택

사용 시점

- SPA/React 기반 웹사이트
- 로그인이 필요한 페이지
- 버튼 클릭 후 데이터 로드
- 파일 다운로드 자동화

3.2 설치

```
# 패키지 설치
pip install playwright

# 브라우저 바이너리 설치 (필수)
playwright install chromium
```

3.3 기본 구조: 브라우저 초기화

```
from playwright.sync_api import sync_playwright

with sync_playwright() as p:
    # 브라우저 실행 옵션
    browser = p.chromium.launch(
        headless=False,      # False: 브라우저 창 표시 (디버깅용)
        slow_mo=300          # 각 동작 사이 300ms 대기 (안정성)
    )

    # 컨텍스트 생성 (다운로드 허용 필수!)
    context = browser.new_context(accept_downloads=True)

    # 페이지 생성 및 이동
    page = context.new_page()
    page.goto("https://www.kblife.co.kr/customer-common/productList.do")
    page.wait_for_timeout(3000)  # 페이지 로딩 대기
```

> **중요:** `accept_downloads=True` 를 설정하지 않으면 파일 다운로드가 차단됩니다.

3.4 요소 탐색 및 데이터 추출

```
# 모든 row 요소 선택
rows = page.locator("div.row")
row_count = rows.count()
print(f"총 row 개수: {row_count}")

# 각 row 순회
for r in range(row_count):
    row = rows.nth(r)

    # row 내부의 cell 텍스트 추출
    # [class^='cell-'] : class가 'cell-'로 시작하는 모든 요소
    cell_texts = row.locator("div[class^='cell-']").all_inner_texts()

    # 다운로드 버튼 찾기
    download_buttons = row.locator("a.downFile")
    btn_count = download_buttons.count()
```

Locator 문법 정리

문법	설명	예시
`page.locator("태그")`	태그로 선택	`page.locator("div")`
`.class`	클래스로 선택	`page.locator("a.downFile")`

`[class^='prefix']`	클래스 접두사로 선택	`div[class^='cell-']`
`.nth(n)`	n번째 요소 선택	`rows.nth(0)`
`.count()`	요소 개수	`buttons.count()`
`.all_inner_texts()`	모든 텍스트 리스트	`cells.all_inner_texts()`

3.5 파일 다운로드 처리

```
# 다운로드 이벤트 대기 + 버튼 클릭
with page.expect_download(timeout=10000) as download_info:
    button.click()

# 다운로드 완료 후 저장
download = download_info.value
filename = f"{product_name}__{sale_period}__{file_type}.pdf"
save_path = DOWNLOAD_DIR / filename
download.save_as(save_path)
print(f"저장 완료: {save_path}")
```

3.6 텍스트 정제 유틸리티 함수

```
import re

def clean_text(text: str) -> str:
    """줄바꿈, 탭, 다중 공백을 단일 공백으로 정리"""
    if not text:
        return ""
    text = re.sub(r'[\r\n\t]+', ' ', text)  # 줄바꿈/탭 → 공백
    text = re.sub(r'\s+', ' ', text).strip() # 다중 공백 → 단일 공백
    return text

def clean_filename(text: str) -> str:
    """파일명에 사용 불가한 문자 제거"""
    text = clean_text(text)
    text = re.sub(r'[\\/*?:"<>|]', '', text)  # 윈도우 금지 문자 제거
    return text[:100] if text else "unknown"  # 길이 제한
```

3.7 판매기간 파싱 (최신 상품 필터링)

```
def parse_sale_period(period_text: str):
    """
    판매기간 텍스트에서 시작일/종료일 추출

    예시:
    - '2026/01/01 ~ 2026/02/08' → ('2026/01/01', '2026/02/08')
    - '2026/02/09 ~'            → ('2026/02/09', '')  ← 현재 판매중!
    """
    text = clean_text(period_text)

    # 시작일 ~ 종료일 패턴
    m_full = re.search(r'(\d{4}/\d{2}/\d{2})\s*~\s*(\d{4}/\d{2}/\d{2})', text)
    if m_full:
        return m_full.group(1), m_full.group(2)

    # 시작일 ~ (종료일 없음 = 현재 판매중)
    m_open = re.search(r'(\d{4}/\d{2}/\d{2})\s*~', text)
    if m_open:
        return m_open.group(1), ""
```

```
    return "", ""

# 사용 예시: 최신 상품만 필터링
sale_start, sale_end = parse_sale_period(sale_period_raw)
is_latest = (sale_start != "" and sale_end == "")  # 종료일 없으면 최신
```

4 pdfplumber PDF 텍스트 추출

4.1 pdfplumber란?

PDF에서 텍스트, 테이블, 메타데이터를 추출하는 Python 라이브러리입니다. PyMuPDF, pdfminer 등 대비 사용이 간편합니다.

4.2 설치

```
pip install pdfplumber
```

4.3 기본 텍스트 추출

```
import pdfplumber
from pathlib import Path

def extract_pdf_text(pdf_path: Path) -> str:
    """PDF에서 전체 텍스트 추출 (페이지 번호 마킹)"""
    texts = []

    with pdfplumber.open(pdf_path) as pdf:
        for i, page in enumerate(pdf.pages):
            text = page.extract_text()
            if text:
                # 공백 정리 + 페이지 번호 마킹
                cleaned = re.sub(r'\s+', ' ', text).strip()
                texts.append(f"[PAGE {i+1}] {cleaned}")

    return "\n".join(texts)
```

페이지 마킹의 이점:

- `[PAGE 1]`, `[PAGE 2]` 형태로 마킹하면 LLM이 문서 구조를 더 잘 이해
- 디버깅 시 어느 페이지에서 정보가 추출되었는지 추적 가능
- 긴 문서에서 특정 섹션 위치 파악 용이

4.4 추출 결과 예시

```
[PAGE 1] KB VIP 변액연금보험 무배당(미보증형) 상품설명서 ...
[PAGE 2] 1. 상품 개요 이 보험은 변액연금보험으로서 ...
[PAGE 3] 2. 가입 자격 계약자: 만 15세 이상 피보험자: 만 0세 ~ 70세 ...
```

5 Pydantic JSON 스키마 설계

5.1 스키마 설계 원칙

1. **계층적 구조화:** 관련 필드를 논리적 그룹(클래스)으로 묶기
2. **Optional 활용:** 문서마다 있을 수도/없을 수도 있는 필드는 `Optional[타입]`
3. **List 타입:** 반복되는 항목(보장내용, 특약 등)은 `List[클래스]` 로
4. **명확한 필드명:** LLM이 이해하기 쉬운 영문 snake_case
5. **기본값 설정:** `Field(default_factory=list)` 로 빈 리스트 기본값

5.2 보험상품 스키마 전체 구조

```
from typing import List, Optional
from pydantic import BaseModel, Field

# ===== 1. 상품 메타데이터 =====
class ProductMetadata(BaseModel):
    insurer: Optional[str] = None           # 보험사명
    product_name: Optional[str] = None      # 상품명
    product_type: Optional[str] = None      # 상품 유형 (연금, 종신, 암보험 등)
    subtype: Optional[str] = None           # 세부 유형 (변액, 저축성 등)
    guarantee_type: Optional[str] = None    # 보증 유형 (무배당, 유배당 등)
    document_type: Optional[str] = None     # 문서 종류 (상품설명서, 약관 등)

# ===== 2. 가입 자격 =====
class Eligibility(BaseModel):
    min_age: Optional[int] = None           # 최소 가입 나이
    max_age: Optional[int] = None           # 최대 가입 나이
    age_text: Optional[str] = None          # 나이 조건 원문
    premium_payment_options: List[str] = Field(default_factory=list)  # 납입 방식
    minimum_premium_text: Optional[str] = None  # 최소 보험료
    eligibility_notes: List[str] = Field(default_factory=list)  # 기타 조건

# ===== 3. 보장 항목 (주계약) =====
class BenefitItem(BaseModel):
    benefit_name: Optional[str] = None      # 보장명
    trigger_condition: Optional[str] = None # 지급 조건
    payout_text: Optional[str] = None       # 지급 금액/비율
    notes: Optional[str] = None             # 비고

# ===== 4. 특약 항목 =====
class RiderItem(BaseModel):
    rider_name: Optional[str] = None            # 특약명
    coverage_summary: Optional[str] = None      # 보장 요약
    trigger_condition: Optional[str] = None     # 지급 조건
    payout_text: Optional[str] = None           # 지급 금액
    subscription_limit_text: Optional[str] = None  # 가입 한도
    waiting_period_text: Optional[str] = None   # 면책/감액 기간
    exclusion_text: Optional[str] = None        # 보장 제외 사항

# ===== 5. 최종 결과 스키마 =====
class InsuranceParseResult(BaseModel):
    product_metadata: ProductMetadata
    eligibility: Eligibility
    main_contract_benefits: List[BenefitItem] = Field(default_factory=list)
    riders: List[RiderItem] = Field(default_factory=list)
```

5.3 스키마 구조 다이어그램

```
InsuranceParseResult ├── product_metadata: ProductMetadata │ ├── insurer │ ├── product_name │ ├── product_type │ ├── subtype │ ├── guarantee_type │ └── document_type │ ├── eligibility: Eligibility │ ├── min_age / max_age / age_text │ ├── premium_payment_options: List[str] │ ├── minimum_premium_text │ └── eligibility_notes: List[str] │ ├── main_contract_benefits: List[BenefitItem] │ └── [benefit_name, trigger_condition, payout_text, notes] │ └── riders: List[RiderItem] └── [rider_name, coverage_summary, trigger_condition, ...]
```

6 LLM 기반 구조화 파싱 (OpenRouter)

6.1 OpenRouter 설정

OpenRouter는 다양한 LLM 모델(GPT-4, Claude, Gemini 등)을 단일 API로 사용할 수 있는 서비스입니다.

```
from openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()  # .env 파일에서 환경변수 로드

client = OpenAI(
    api_key=os.getenv("OPENROUTER_API_KEY"),
    base_url="https://openrouter.ai/api/v1"  # OpenRouter 엔드포인트
)
```

보안 주의: API 키는 절대 코드에 직접 작성하지 마세요!

- `.env` 파일에 저장: `OPENROUTER_API_KEY=sk-...`
- `.gitignore` 에 `.env` 추가하여 Git 커밋 방지

6.2 프롬프트 설계

```
prompt = f"""
You are an insurance document parser.

Return ONLY one valid JSON object.
Do not add markdown.
Do not add explanations.
Do not wrap in triple backticks.

Required top-level keys:
- product_metadata
- eligibility
- main_contract_benefits
- riders

Schema guidance:
product_metadata:
  insurer, product_name, product_type, subtype, guarantee_type, document_type

eligibility:
  min_age, max_age, age_text, premium_payment_options,
```

```
  minimum_premium_text, eligibility_notes

main_contract_benefits:
  list of objects with benefit_name, trigger_condition, payout_text, notes

riders:
  list of objects with rider_name, coverage_summary, trigger_condition,
  payout_text, subscription_limit_text, waiting_period_text, exclusion_text

If unknown, use null or empty list.

Document:
{document_text[:80000]}
"""
```

6.3 프롬프트 설계 핵심 포인트

포인트	예시	이유
출력 형식 명시	"Return ONLY one valid JSON object"	불필요한 설명 방지
금지 사항	"Do not add markdown"	파싱 오류 방지
스키마 구조 제공	필드명 명시적 나열	LLM 가이드
누락 처리	"If unknown, use null"	일관된 출력 보장
토큰 제한	`[:80000]`	컨텍스트 초과 방지

6.4 API 호출

```
response = client.chat.completions.create(
    model="openai/gpt-4o-mini",  # 비용 효율적인 모델
    messages=[
        {
            "role": "system",
            "content": "You extract structured insurance data as strict JSON."
        },
        {
            "role": "user",
            "content": prompt
        }
    ],
    temperature=0  # 일관된 출력을 위해 0 설정
)

raw_text = response.choices[0].message.content
```

> **temperature=0 설정 이유:**
> 구조화 파싱은 창의성이 아닌 정확성이 중요합니다. temperature=0으로 설정하면 동일한 입력에 대해 일관된 출력을 얻을 수 있습니다.

7 JSON 후처리 및 검증

7.1 왜 후처리가 필요한가?

LLM은 프롬프트에서 "markdown 쓰지 마"라고 해도 종종 다음과 같이 응답합니다:

```
```json
{
  "product_metadata": { ... }
}
```

Here is the extracted data...
```

이런 응답을 안정적으로 처리하기 위해 후처리 함수가 필수입니다.

7.2 JSON 추출 함수

```
import json
import re

def extract_json_from_text(raw_text: str) -> dict:
    """LLM 응답에서 JSON 객체만 안전하게 추출"""

    if not raw_text or not raw_text.strip():
        raise ValueError("LLM 응답이 비어 있습니다.")

    text = raw_text.strip()

    # Step 1: ```json ... ``` 마크다운 제거
    text = re.sub(r"^```json\s*", "", text, flags=re.IGNORECASE)
    text = re.sub(r"^```\s*", "", text)
    text = re.sub(r"\s*```$", "", text)

    # Step 2: 첫 번째 { 부터 마지막 } 까지 추출
    start = text.find("{")
    end = text.rfind("}")

    if start == -1 or end == -1 or end <= start:
        raise ValueError(f"JSON 객체를 찾지 못했습니다.\n원본: {text[:500]}")

    json_candidate = text[start:end + 1]

    # Step 3: JSON 파싱 시도
    try:
        return json.loads(json_candidate)
    except json.JSONDecodeError as e:
        raise ValueError(f"JSON 파싱 실패: {e}\n후보: {json_candidate[:500]}")
```

7.3 Pydantic 검증

```
# JSON dict → Pydantic 모델로 검증
data = extract_json_from_text(raw_text)
result = InsuranceParseResult.model_validate(data)

# 검증 통과 후 JSON 저장
json_output = result.model_dump_json(indent=2)
```

7.4 디버깅을 위한 원본 저장

```
# LLM 원본 응답 저장 (파싱 실패 시 디버깅용)
raw_output_path = OUTPUT_DIR / "raw_llm_output.txt"
raw_output_path.write_text(raw_text, encoding="utf-8")
```

8 실행 환경 설정 및 명령어

8.1 환경 설정

```
# 1. 가상환경 생성 및 활성화
python -m venv .venv
.venv\Scripts\activate          # Windows
source .venv/bin/activate       # Mac/Linux

# 2. 의존성 설치
pip install playwright pdfplumber pydantic openai python-dotenv

# 3. Playwright 브라우저 설치
playwright install chromium

# 4. 환경변수 파일 생성
# .env 파일 내용:
OPENROUTER_API_KEY=sk-or-v1-xxxxxxxxxxxxxxxx
```

8.2 실행 순서

```
# Step 1: 상품 메타데이터 수집 (선택)
python kb_export_rows.py
# → 결과: kb_rows_metadata.csv

# Step 2: PDF 다운로드
python kb_download_one.py
# → 결과: downloads_latest/*.pdf

# Step 3: PDF 파싱
python parse_one_pdf.py
# → 결과: outputs/parsed_insurance.json
```

8.3 .gitignore 설정

```
# 환경변수 (API 키 보호)
.env
.env.local
.env.*

# Python 가상환경
.venv/
venv/

# 다운로드/출력 데이터
downloads/
downloads_latest/
outputs/

# Python 캐시
__pycache__/
*.pyc
```

8.4 출력 결과 예시

```
{
  "product_metadata": {
    "insurer": "KB생명",
    "product_name": "KB VIP 변액연금보험 무배당(미보증형)",
    "product_type": "연금보험",
    "subtype": "변액",
    "guarantee_type": "무배당",
    "document_type": "상품설명서"
  },
  "eligibility": {
    "min_age": 15,
    "max_age": 70,
    "age_text": "계약자 만 15세 이상, 피보험자 만 0세~70세",
    "premium_payment_options": ["일시납", "월납", "연납"],
    "minimum_premium_text": "월 30만원 이상",
    "eligibility_notes": []
  },
  "main_contract_benefits": [...],
  "riders": [...]
}
```


기술 스택 요약

기술	버전	용도	핵심 기능
Playwright	1.40+	웹 자동화	동적 페이지 크롤링, 파일 다운로드
pdfplumber	0.10+	PDF 처리	텍스트 추출, 페이지별 처리
Pydantic	2.0+	데이터 검증	JSON 스키마 정의, 타입 검증
OpenAI SDK	1.0+	LLM API	OpenRouter를 통한 GPT-4o-mini 호출
python-dotenv	1.0+	환경변수	.env 파일에서 API 키 로드

TrackB Project - KB생명 보험상품 데이터 수집 및 구조화 파이프라인

문서 작성일: 2026-03-23 | 대상: 학회원 기술 전달용